Residual Graph Indexing: Auditing and Repairing Index-Time Information Loss in Community-Based GraphRAG

Jeyaviknan SR[1,*], Adithya DS[1], Aatreye Dev[1], and Karthika Veeramani[1]

***Abstract*—GraphRAG answers corpus-level questions by reading summaries of graph communities instead of the graph itself. A relationship whose evidence never enters the input of those summaries cannot reach an answer through any retrieval policy that reads them, however well that policy ranks. We separate two strengths of this loss: a relationship is out of scope when no input about its own community carries it, and its evidence is absent when no selected input carries it in any form. Running GraphRAG 3.1.1's own indexing code, without calling a large language model (LLM), on all 20 novels of GraphRAG-Bench Novel, we find that at the default query level the summary inputs keep 12.4% of 116,217 relationships in scope. Evidence is rarely absent at that level, but at the root level 87.6% of relationships have no evidence in any input. In the standard pipeline, the input builder admits only 4.6% to 16.3% of within-community relationships because it keeps one relationship per entity, whatever the budget. We propose Residual Graph Indexing (RGI), which keeps GraphRAG's summaries and search and adds a residual in the video-coding sense: an exact, audited record of what the summaries leave out. At the same 8,000-token budget and with no extra LLM calls, RGI raises in-scope coverage from 12.4% to 34.6% (mean over novels +24.6 points, 95% confidence interval (CI) [20.5, 28.3]) without dropping anything GraphRAG kept. Answer-level effects are shown only in simulation; we give the pre-registered protocol that will test them.**

***Index Terms*—retrieval-augmented generation, knowledge graphs, GraphRAG, community detection, summarization, information loss, indexing**

I. Introduction

RETRIEVAL-augmented generation (RAG) grounds a language model's answer in retrieved text [1], [2]. Chunk retrieval works when the evidence sits in a few passages. It works poorly for questions about a whole corpus, such as the main conflicts in a novel or the themes running through a collection of reports. GraphRAG [3] was built for those questions. A large language model (LLM) extracts entities and relationships from the corpus, the Leiden algorithm [4] groups the resulting graph into a hierarchy of communities, and an LLM writes one summary per community. Global search then maps the question over the summaries at one level of the hierarchy and reduces the partial answers into one.

Global search never reads the graph. It reads the summaries, and each summary is written from a *summary input*: a token-limited selection of the community's content. Content that does not enter any input can appear in a later answer only through the model's prior knowledge. This happens while the index is built, before any question exists. A better retriever can change which summaries are read; it cannot change what they contain.

This paper measures that loss and then tries to repair it. Reading the code of GraphRAG 3.1.1 [5], we found five mechanisms behind it (Section III, Fig. 1). The most visible one affects relationships whose two entities were put in different communities, since such a relationship is in no community's scope. Inputs longer than the 8,000-token budget are cut where the first overflowing item begins. Whatever the budget, the standard input builder passes on at most one relationship per entity, and this behavior has also been reported on the project's issue tracker [6]. A parent community that is too large gets summarized from the summaries of its children instead of from its own content, and entities lying outside the largest connected component (LCC) are never clustered at all. Querying at a coarser level removes boundaries, but the same budget then has to carry more content, so the loss moves from one mechanism to another as the level changes.

We then propose *Residual Graph Indexing* (RGI). We borrowed the idea from video coding, where a codec stores keyframes and, alongside them, residuals holding whatever the keyframes failed to capture [7]. In RGI the community summaries play the part of the keyframes, and GraphRAG writes them exactly as before. What we call the residual is the set of relationships that the summary inputs leave out, and we compute it exactly from the index tables GraphRAG already produces. RGI changes how each input spends its budget, adds bridge records for relationships that cross communities, keeps the remainder in a residual store that query time can use without adding tokens, and checks at build time that every relationship is accounted for.

Our contributions are the following.

- A loss model for community-summary GraphRAG that separates relationships that are out of scope from relationships whose evidence is absent from every input. It comes with three propositions: absent evidence is unreachable for every summary-reading retrieval policy, boundary loss can only grow toward the leaves of a nested hierarchy, and the standard input builder admits at most $2/\bar{d}$ of a community's internal relationships, where $\bar{d}$ is their mean internal degree (Section III).

*Corresponding author.

[1]Jeyaviknan SR, Adithya DS, Aatreye Dev and Karthika Veeramani are with the School of Computer Science and Engineering, Vellore Institute of Technology, Chennai 600127, India (e-mail: jeyaviknan.sr2023@vit.ac.in; adithya.ds2023@vitstudent.ac.in; aatreye.dev2023@vitstudent.ac.in; karthika.v@vit.ac.in).

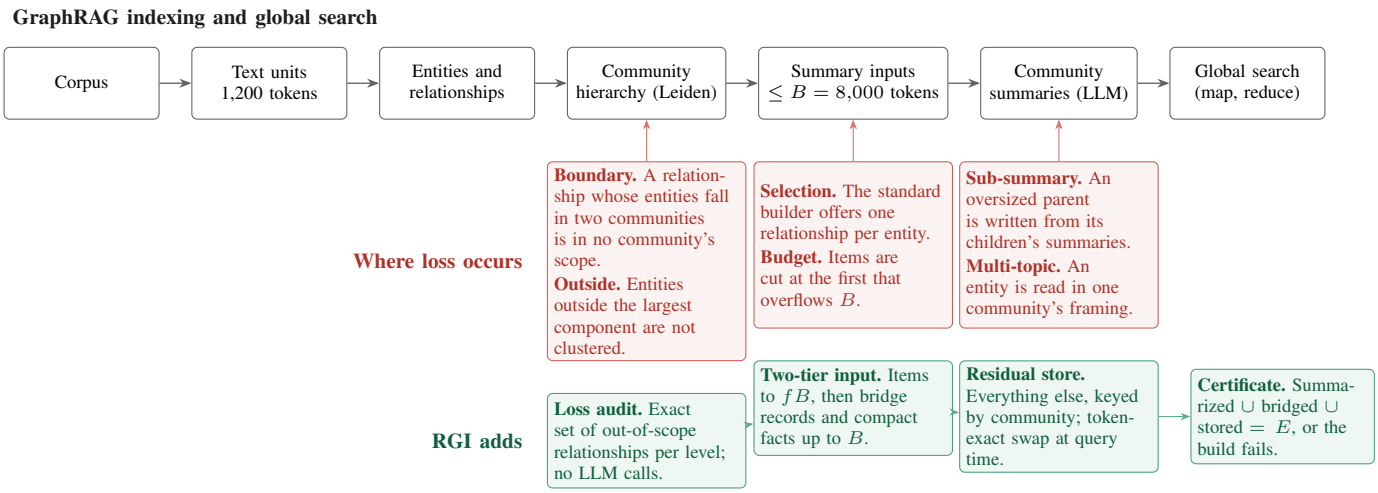


Fig. 1. GraphRAG's indexing pipeline, the six places where content leaves the path to an answer, and the four components RGI adds. Summaries, clustering and retrieval are unchanged.

- An exact census, run without an LLM through GraphRAG 3.1.1's own code, of all 20 novels in GraphRAG-Bench Novel [8] (Sections V and VI).
- RGI, with four guarantees that we prove and then check on every run: no input exceeds its budget, no relationship that GraphRAG kept in scope is dropped, the number of LLM calls is unchanged, and every relationship is certified as summarized, bridged or stored (Section IV).
- A controlled simulation that carries the loss through retrieval to judged answers, and a protocol for the answer-level test whose kill thresholds were committed to version control before any data existed (Sections VI-F and IX).

We state plainly what this paper does not show. No experiment here measures answer accuracy on an LLM-built index. The census uses GraphRAG's LLM-free fast pipeline with a substitute entity extractor, and the simulation is synthetic. Section VIII discusses each limit.

II. Background and Related Work

A. Community-summary GraphRAG

GraphRAG [3], [5] splits documents into text units (1,200 tokens with 100 tokens of overlap by default, counted with the `o200k_base` encoding). Its standard pipeline asks an LLM to extract entities and relationships with descriptions. Its fast pipeline extracts noun phrases instead, links phrases that occur in the same text unit, weights the links by pointwise mutual information (PMI) and prunes them. Both pipelines cluster the graph with hierarchical Leiden [4] as implemented in graspologic [9], splitting any community larger than ten entities. Each community then receives a report written by an LLM from a summary input of at most 8,000 tokens (`max_input_length`). Global search at level ℓ (the command-line default is $\ell = 2$) gives every entity the report of its deepest community at or above level ℓ and maps the question over those reports. LazyGraphRAG [10] defers summarization to query time.

The original evaluation compared global search at several hierarchy levels with a map-reduce summarization over the source text, which has no partition, and found no significant difference in aggregate answer quality [3]. Later evaluations report that graph-based RAG does not uniformly beat chunk retrieval [8], [11], [12], and GraphRAG-Bench reports that global search uses 331,375 tokens per query on its Novel corpus [8]. We return to the first finding in Section VII, because an aggregate quality score can hide element-level loss.

B. Other graph-based retrieval

LightRAG [13] retrieves entities and relationships directly at two levels of abstraction and has no community layer. HippoRAG [14] runs personalized PageRank over an extracted graph. RAPTOR (recursive abstractive processing for tree-organized retrieval) [15] builds a tree of recursive summaries over chunks. G-Retriever [16] retrieves subgraphs of textual graphs. ArchRAG [17] detects communities from links and attributes and indexes them hierarchically. Surveys cover the wider field [18].

C. Work on community boundaries

Several recent systems target the gaps between communities, mostly at query time (Table I). HiRAG [19] builds a layered knowledge hierarchy and retrieves the knowledge that connects local entities with global communities. LeanRAG [20] adds explicit relations between aggregated summary nodes, which it calls "semantic islands" when unlinked, and retrieves along lowest-common-ancestor paths. Deep GraphRAG [21] filters across communities, refines within them, and searches entities inside the chosen ones. OMD-GraphRAG (ontology-guided extraction, multi-dimensional clustering and dual-channel fusion) [22] is the closest to our bridge records: an external entity is projected into a community's summary representation when the share of its edges that point into the community exceeds a threshold. Hossain and Sarıyüce [23] replace Leiden with k-core decomposition, prove that modularity admits exponentially many near-optimal partitions on sparse knowledge graphs, and sample within a token budget.

TABLE I
HOW RELATED SYSTEMS TREAT THE SPACE BETWEEN COMMUNITIES

System	Acts at	Mechanism
GraphRAG [3]	index	One summary per community; cross-community relationships are out of every scope
HiRAG [19]	index, query	Layered hierarchy; retrieval of connecting knowledge between layers
LeanRAG [20]	index, query	Explicit relations among aggregated nodes; path retrieval
Deep GraphRAG [21]	query	Inter-community filtering, then intra-community search
OMD-GraphRAG [22]	index	Threshold-based projection of boundary entities into summaries
Core hierarchies [23]	index	k-core hierarchy replaces Leiden
RGI (ours)	index, query	Exact loss audit; budget-neutral bridges and compact facts; residual store; coverage certificate

RGI differs in where it starts. It changes neither the clustering nor the retrieval nor the summary prompt. It begins from the exact set of relationships the inputs leave out, spends a fixed share of the existing budget on the most connected of them, and accounts for every relationship. To our knowledge none of the systems above reports an element-level account of what the summaries omit.

D. Compression, granularity and community structure

RAG systems already trade detail for budget. RECOMP (retrieve, compress, prepend) compresses retrieved documents before generation [24], Dense X retrieval studies the unit of retrieval [25], and long contexts are used unevenly by the model [26]. That last result is why every comparison in this paper holds the token count fixed: content added on top of a context confounds the content with its length. On the clustering side, modularity has a resolution limit [27] and many near-equivalent optima [28], [29], and some entities connect several modules by nature [30]. Partition boundaries are therefore partly arbitrary, which makes it more important to know what they cut.

III. A MODEL OF INDEX-TIME LOSS

A. Setting

The corpus is split into text units $\mathcal{U}$. Extraction yields a graph $G = (V, E)$, and each relationship $r = (a, b) \in E$ has a support $U(r) \subseteq \mathcal{U}$, the units it was extracted from. Clustering yields communities $\mathcal{C}$. Each $c \in \mathcal{C}$ has a level $\lambda(c)$, with 0 the coarsest, and the hierarchy is nested: the children of a community partition its entities. Global search at level ℓ reads the summaries of

$$S_\ell = \{\sigma_\ell(v) : v \in V\}, \quad \sigma_\ell(v) = \arg \max_{c \ni v,\ \lambda(c) \le \ell} \lambda(c), \quad (1)$$

so every entity is read through its deepest community at or above level ℓ; $\sigma_\ell(v)$ is undefined for an entity that was never clustered. GraphRAG implements this roll-up by taking, for each entity, the largest community identifier at levels up to ℓ. We verified on every census graph that this rule returns the deepest community. Each c has a summary input $I(c) = \beta(c; B)$ built by an input builder β under budget B, and a summary $s(c) = M(I(c))$ written by a model M. We write $T(x)$ for the number of tokens in a string x under the `o200k_base` encoding, so $T(I(c)) \le B$. Table II collects every abbreviation and symbol used in the paper.

TABLE II
ABBREVIATIONS AND SYMBOLS

Abbreviations	
CI	confidence interval
G1 to G4	the four pre-registered kill criteria (Table XI)
LCC	largest connected component
LLM	large language model
NLI	natural language inference
NLTK	Natural Language Toolkit
PMI	pointwise mutual information
RAG	retrieval-augmented generation
RGI	Residual Graph Indexing, the method of this paper
Graph and index	
$G = (V, E)$	extracted graph with entities V and relationships E
$r = (a, b)$	relationship between entities a and b
$\mathcal{U}$, $U(r)$	text units; the units that support r
$\mathcal{C}$, c, $\lambda(c)$	communities; one community; its level (0 is the coarsest)
ℓ	query level of global search
$\sigma_\ell(v)$, S_ℓ	community through which v is read at level ℓ; all such communities
$I(c)$, β, B	summary input of c; input builder; budget in tokens
$s(c)$, M	summary of c; summarizing model
$T(x)$	number of tokens in x under `o200k_base`
L_ℓ	relationships out of scope at level ℓ
R_ℓ^G, R_ℓ^{RGI}	relationships in scope and represented under GraphRAG and under RGI
$\bar{d}_{\mathrm{in}}(c)$	mean number of internal relationships per entity of c
π, $C_\pi(q)$	retrieval policy; its context for query q
RGI parameters and Algorithm 1	
f, b, m	full-item cutoff, bridge share, minimum support
E_{in}, E_{bd}	in-scope and boundary relationships of c
P, $P_{1..k}$, k_G	GraphRAG's items for c; their first k; the prefix length GraphRAG admits
X, $\oplus$, $\mathrm{rep}(X)$	input under construction; append; relationships represented in X
$\prec$	priority order, higher combined degree first
Evaluation	
$Y(q)$	gold relationships of question q
K	summaries read per question (simulation)
κ	Cohen's agreement coefficient

A relationship is *in scope* for c if both entities belong to c. It is *represented* in $I(c)$ if its record appears there (standard pipeline) or one of its supporting text units does (fast pipeline).

Definition 1 (Status)**.** *At level ℓ every $r = (a, b) \in E$ has exactly one status. It is* outside *if $\sigma_\ell(a)$ or $\sigma_\ell(b)$ is undefined, and* boundary *if $\sigma_\ell(a) \neq \sigma_\ell(b)$. Otherwise let $c = \sigma_\ell(a) = \sigma_\ell(b)$. Then r is* represented *if it is represented in $I(c)$,* sub-summary *if $I(c)$ was assembled from the summaries of c's children,* selection *if β never offered r, and* budget *if β offered r and cut it at B.*

Out of scope at level ℓ means any status other than represented, and $L_\ell \subseteq E$ denotes the set of relationships

out of scope at level ℓ. Scope is a property of the index structure. Evidence is a separate question, because one text unit can support many relationships and an entity description can paraphrase relationships it took part in. We therefore grade every relationship by the strongest way its evidence reaches a selected input: *in scope*; *endpoint*, when a supporting unit sits in the input of a community that holds one of its entities; *elsewhere*, when a supporting unit sits only in inputs about other entities; and *absent*, when no record and no supporting unit appears in any input in S_ℓ. A sixth kind of loss is entity-level: an entity whose text units span several topics is filed under one community and read in that framing only. We call it *multi-topic* loss. It does not remove the entity from the index, so it falls outside the propositions below.

B. Three propositions

Proposition 1 (Absent evidence is unreachable)**.** *Fix a level ℓ and any retrieval policy π whose context $C_\pi(q)$ for a query q is a function of q and $\{s(c) : c \in S_\ell\}$. If the evidence of r is absent at level ℓ, then $C_\pi(q)$ does not change when the units in $U(r)$ are edited, for every q.*

Proof. The corpus can influence a summary $s(c)$ only through its input $I(c)$. If r's evidence is absent, no unit of $U(r)$ and no record derived from it appears in any $I(c)$ with $c \in S_\ell$, so editing those units leaves every input, every summary and therefore $C_\pi(q)$ unchanged. □

Proposition 1 is deliberately narrow. It covers absent evidence, not everything out of scope. A relationship that is out of scope but whose supporting text sits in some input can still be mentioned by a summary, and a description of an entity can mention a relationship that has no record of its own. How often that happens is an empirical question about the summarizer. Gate G1, the first of the four pre-registered kill criteria G1 to G4 in Section IX, is designed to measure it.

Proposition 2 (Boundary loss grows toward the leaves)**.** *On a nested hierarchy, if $\ell < \ell'$ then every relationship with status boundary at level ℓ has status boundary at level ℓ'.*

Proof. Two distinct communities of a nested hierarchy are either disjoint or one contains the other. Suppose $\sigma_\ell(a) \neq \sigma_\ell(b)$ and $\sigma_\ell(a) \supsetneq \sigma_\ell(b)$. Then $\sigma_\ell(a)$ has children, the child holding a sits at a level no deeper than $\lambda(\sigma_\ell(b)) \leq \ell$, and it is deeper than $\sigma_\ell(a)$, which contradicts the choice of $\sigma_\ell(a)$. So $\sigma_\ell(a)$ and $\sigma_\ell(b)$ are disjoint. For every clustered v, $\sigma_{\ell'}(v) \subseteq \sigma_\ell(v)$, hence $\sigma_{\ell'}(a)$ and $\sigma_{\ell'}(b)$ are disjoint as well. □

Coarser levels pay for fewer boundaries with larger inputs. With a fixed B, the share of inputs that overflow, and that must be truncated or assembled from child summaries, can only rise as communities merge. Proposition 2 and this budget effect pull in opposite directions, which is the trade-off the census measures.

Proposition 3 (Selection bound)**.** *In GraphRAG 3.1.1's standard input builder, at most $|c|$ of the relationships in scope for a community c can be represented in $I(c)$, whatever B is. The represented share of c's internal relationships is therefore at most $2/\bar{d}_{\text{in}}(c)$, where $\bar{d}_{\text{in}}(c)$ is the mean number of internal relationships per entity of c.*

Proof. Inside `_prepare_reports_at_level`, the relationships of the level are grouped twice, once by source entity and once by target entity. Only the first relationship in each group survives, and the two resulting tables are merged with `combine_first`, so an entity contributes one relationship record at most and $I(c)$ can offer no more than $|c|$ records. Because c has $|c|\,\bar{d}_{\text{in}}(c)/2$ internal relationships, the represented share cannot exceed $2/\bar{d}_{\text{in}}(c)$. □

In practice the real share is lower still. The one record an entity contributes may point to an entity in another community, and two entities can end up contributing the same record. With an internal degree close to 2, as in a sparse graph, the bound hardly matters, whereas on a dense graph the cap removes most of the relationships.

IV. Residual Graph Indexing

A. Requirements

RGI was designed under four requirements. The first two keep the comparison fair. GraphRAG's clustering, summary prompt and global search stay exactly as they are, and no input or query context gets a larger token budget, because extra tokens alone change how a model behaves [26]. With both held fixed, any difference in results has to come from what the inputs contain. The other two concern cost and accounting: indexing may not need any extra LLM call, and the method has to be able to report, for each relationship, where it ended up. Fig. 1 shows the four components we built to meet these requirements.

B. Loss audit

For each level ℓ, the audit gives every relationship one of the statuses of Definition 1. Finding the boundary and outside relationships needs only a set difference on tables GraphRAG already writes, namely all relationship identifiers minus the ones listed inside the selected communities. The budget and selection statuses are obtained by replaying the input builder with GraphRAG's own sort key, budget and tokenizer, and because the builder is deterministic the replay is exact. No model is called during the audit, and apart from tokenization its cost per level grows linearly with $|E|$.

C. Two-tier summary input

GraphRAG adds items to an input in priority order until one of them would overflow B. It stops there, and every remaining item is dropped together. RGI uses the same items in the same order and differs only in how it spends the last part of the budget (Algorithm 1, Fig. 2). An input that already fits within B is kept whole, and whatever slack is left goes to bridge records. For an input that overflows, RGI keeps full items only up to a cutoff fB and fills the space after it with one-line *compact facts* of the form `source,target,support`, in GraphRAG's own priority order (highest combined degree first).

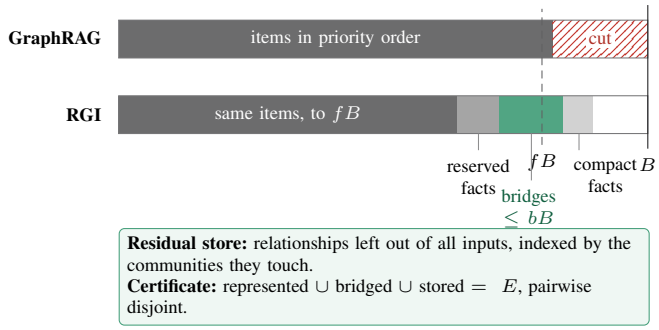


Fig. 2. Layout of one summary input. GraphRAG adds items until the first one that overflows B and drops everything from there on. RGI stops full items at fB. In the space that remains it first writes compact facts for whatever the shorter prefix lost, then bridge records, then further compact facts, and the total stays within B.

A compact fact costs about ten tokens and a text unit about 1,200. What facts give up is detail, since a fact records only that two entities are related and says nothing about the relation itself. The first facts RGI writes are reserved for relationships that GraphRAG's own input represented and that the shortened prefix has lost. Should these reserved facts not fit, RGI keeps GraphRAG's own input for that community and uses only the leftover slack; the monotonicity guarantee below depends on this rule. For communities whose input is assembled from child summaries, the child summaries are capped at fB instead of B, and the tail holds compact facts about the community's own relationships.

Algorithm 1 uses the following notation. P is GraphRAG's list of items for c in its own order, $P_{1..k}$ its first k items, and k_G the length of the prefix GraphRAG admits under B. X is the input under construction and $\oplus$ appends to it. $\mathrm{rep}(X)$ is the set of relationships represented in X; $\mathrm{fact}(r)$, $\mathrm{facts}(Q)$ and $\mathrm{bridge}(r)$ render one-line records for a relationship r or a set Q; *pend* holds the in-scope relationships still waiting for a compact fact; and $r \prec r'$ means that r has the higher combined degree.

Algorithm 1 RGI summary input for community c at level ℓ

Require: items P in GraphRAG order; in-scope set E_{in}; boundary set E_{bd}; budget B; cutoff f; bridge share b; support m; priority $\prec$

$k_G \leftarrow$ GraphRAG's admitted prefix length under B
if $T(P) \leq B$ **then** $X \leftarrow P$; *pend* $\leftarrow \emptyset$
else
  $k \leftarrow$ longest prefix with $T(P_{1..k}) \leq fB$ ▷ $k \leq k_G$
  $Q \leftarrow \mathrm{rep}(P_{1..k_G}) \setminus \mathrm{rep}(P_{1..k})$ sorted by $\prec$ ▷ reserved
  if $T(P_{1..k} \oplus \mathrm{facts}(Q)) > B$ **then** $k \leftarrow k_G$; $Q \leftarrow \emptyset$ ▷ fallback
  $X \leftarrow P_{1..k} \oplus \mathrm{facts}(Q)$; *pend* $\leftarrow E_{\mathrm{in}} \setminus \mathrm{rep}(P_{1..k_G})$
for $r \in E_{\mathrm{bd}}$ with $|U(r)| \geq m$, in $\prec$ order **do**
  if bridge tokens would exceed $\min(bB,\ B - T(X))$ **then break**
  append $\mathrm{bridge}(r)$ to X
for $r \in$ *pend* in $\prec$ order **do**
  if $T(X \oplus \mathrm{fact}(r)) > B$ **then break**
  append $\mathrm{fact}(r)$ to X
while $T(\mathrm{render}(X)) > B$ **do** drop the last fact of X
residual$(c) \leftarrow (E_{\mathrm{in}} \cup E_{\mathrm{bd}}) \setminus \mathrm{rep}(X)$
return X

D. Bridge records

Each community spends at most a share b of B on *bridge records*: boundary relationships with at least m supporting text units, ranked by combined degree and written as compact facts into the inputs of both endpoint communities. A relationship counts as bridged if it lands in at least one of the two. Bridges use the description GraphRAG already extracted when one exists, so no summary is rewritten and no model is called.

E. Residual store and query-time substitution

Every relationship that is neither represented nor bridged goes to a residual store keyed by the communities it touches (or by a global key, for outside relationships). After global search selects its reports, RGI can add the stored facts of those communities that best match the question. Added facts *replace* the lowest-ranked context so that the token count of the context stays exactly the same. Our implementation of this swap holds token counts equal under `o200k_base` on 50 random contexts, 10 contexts with mixed scripts and punctuation, and five insertion sizes from 5% to 95% of the context.

F. Coverage certificate

At build time RGI checks, for every level, that the represented, bridged and stored sets are disjoint and that their union is E. A failure stops the build. GraphRAG today discards content without a record; the certificate makes the discard visible and bounded, and it would also flag loss from a mechanism nobody anticipated, such as the selection behavior of Proposition 3.

G. Guarantees

Let R_ℓ^G and R_ℓ^{RGI} be the relationships in scope and represented under GraphRAG and under RGI at level ℓ.

Proposition 4 (Guarantees)**.** *For every community and level: (i) $T(I_{\mathrm{RGI}}(c)) \leq B$; (ii) $R_\ell^G \subseteq R_\ell^{\mathrm{RGI}}$; (iii) RGI makes exactly one summarization call per community, as GraphRAG does; (iv) the represented, bridged and stored sets partition E.*

Proof. (i) The final loop of Algorithm 1 enforces the bound on the rendered string with the production tokenizer. (ii) If the input fits, X contains all of P. Otherwise either the reserved facts Q restore every relationship that $P_{1..k_G}$ represented and $P_{1..k}$ did not, or the fallback restores $P_{1..k_G}$ itself; bridges and later facts only add. Reserved facts precede all other appended lines, so the final loop reaches them only if $P_{1..k} \oplus \mathrm{facts}(Q)$ alone overflows. The fallback test excludes that case up to tokenizer merges at line boundaries, and we check the outcome on every run. (iii) RGI changes inputs, not the number of summaries. (iv) Each relationship is placed in the residual store exactly when it is neither represented nor bridged. □

The guarantee in (ii) is about relationships in scope. When an input overflows, RGI swaps GraphRAG's last full items for compact facts, so the text of those items is no longer in the input, and a relationship that was only incidentally exposed through that text can lose its exposure. The census counts these cases.

V. Experimental Setup

A. Corpus

GraphRAG-Bench [8] provides two corpora. We use Novel: 20 pre-twentieth-century novels from Project Gutenberg, chosen by the benchmark's authors to limit overlap with pretraining data, totalling 4,819,610 characters and 1,217,195 tokens. It comes with 2,010 questions in four types (fact retrieval, complex reasoning, contextual summarization, creative generation), each with gold evidence given as sentences and as $(\textit{head}, \textit{relation}, \textit{tail})$ triples. The data are released under the MIT license.

B. Index construction

We index each novel separately with GraphRAG 3.1.1's fast pipeline at its shipped defaults (Table III). Every stage except one calls GraphRAG's own functions: the token chunker, the co-occurrence edge builder with PMI weights, `prune_graph` (minimum frequency 2, minimum edge weight at the 40th percentile, removal of the single most connected entity), hierarchical Leiden with maximum community size 10 and the default seed, and the text-unit input builder with its degree ordering and first-overflow cut. The exception is noun-phrase extraction. GraphRAG's default extractor needs tagger data from the Natural Language Toolkit (NLTK) that we could not download on our network, so we substitute a deterministic proper-noun extractor that keeps the default extractor's output format, stop words and token rules (Section VIII). No LLM is called anywhere; a full run over the corpus, including every configuration reported here, takes 149 seconds on an Apple M2 laptop.

TABLE III
Census corpus and index (GraphRAG 3.1.1 fast pipeline)

Novels / questions	20 / 2,010
Text units (1,200 tokens, 100 overlap)	1,024
Extracted entities / relationships	14,825 / 533,441
After pruning: entities / relationships	4,831 / 116,217
Mean degree (pooled; median over novels)	48.1 (31.1)
Communities, all levels	1,226
Deepest level reached	5
Summary input budget B	8,000 tokens (`o200k_base`)
Largest novel's share of relationships	34%
LLM calls	0

To make sure we measure GraphRAG's behavior and not our reading of it, we compared our admission logic with GraphRAG's own `sort_context` on 139 overflowing communities; the admitted text was identical in every case. The standard pipeline's input builder is run unchanged in Section VI-B.

C. Measurements

For each novel, level $\ell \in \{0, 1, 2, 3\}$ and budget $B \in \{4\text{k}, 8\text{k}, 16\text{k}, 32\text{k}\}$ we record each relationship's status (Definition 1) and evidence grade under GraphRAG and under RGI. A census counts every relationship, so we report counts and proportions without p-values. Because one novel holds 34% of all relationships, we report both the pooled share and the mean of per-novel shares, the latter with a 95% bootstrap confidence interval (CI) over novels (10,000 resamples) [31].

D. Gold evidence

We link gold triples to the graph with the parameters frozen before any data were collected: both head and tail must reach a `token_set_ratio` of at least 85 against an entity title of the question's own novel, and a triple whose endpoints link identifies the relationships between them. For a question q we write $Y(q) \subseteq E$ for the relationships its triples resolve to, and q enters the gold analysis when $Y(q)$ is not empty. The quantity of interest is the share of such questions with $Y(q) \cap L_\ell \neq \emptyset$, that is, with at least one gold relationship out of scope. Our pre-registration also froze a rule: if more than 15% of triples fail to link, the gold-intersection gate (G2 in Section IX) is reported as inconclusive. We apply that rule here.

E. RGI configuration

The defaults are $f = 0.8$, $b = 0.12$ and $m = 2$. The first two were fixed in our design document before any real-data run; m was fixed before the full census. Section VI-E reports every value we tried.

VI. Results

A. How much each level loses

The census results are in Table IV and Fig. 3. At the default level 2, GraphRAG's inputs keep only 12.4% of relationships in scope, and boundaries account for 84.9%. Almost nothing is lost to the budget at this level, because nearly every community there is a leaf with ten entities or fewer and its input fits. The root level looks very different. Only 1.9% stays in scope there, 47.2% crosses a boundary, and 50.9% of relationships belong to communities whose inputs are built from child summaries. From level 0 to level 3, boundary loss rises monotonically, which is what Proposition 2 predicts.

TABLE IV
Share of relationships (%) by status at $B = 8{,}000$, pooled over novels. Bnd. = boundary, Sub. = sub-summary only, Bud. = budget. The last column is the mean over novels of RGI's in-scope share minus GraphRAG's, with a 95% bootstrap interval. Level 2 is the default.

	GraphRAG (pooled)				RGI	Mean gain
Level	Scope	Bnd.	Sub.	Bud.	Scope	[95% CI]
0	1.9	47.2	50.9	0.00	**12.4**	+25.2 [16.3, 35.4]
1	7.8	71.8	20.4	0.00	**27.4**	+25.5 [20.6, 30.1]
2	12.4	84.9	2.6	0.01	**34.6**	+24.6 [20.5, 28.3]
3	12.1	87.9	0.0	0.01	**35.6**	+24.9 [20.9, 28.6]

Scope is not the same as evidence (Table V). In the fast pipeline an input is made of text units, and one unit names many entities. At level 2, 88.2% of relationships have a supporting unit in the input of a community that holds one of their entities, and only 0.4% have no evidence in any input. At level 0 the inputs of oversized communities hold no raw text at all, and 87.6% of relationships have no evidence

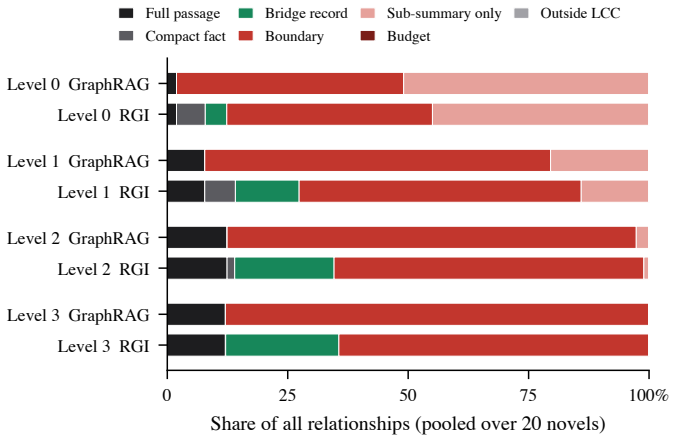


Fig. 3. Where every relationship ends up at each query level, pooled over 20 novels. Black, grey and green segments are in scope and red segments are out of scope. LCC = largest connected component. Since RGI leaves the clustering alone and only changes how inputs use their budget, the boundary segment shrinks only by the relationships that bridge records carry.

anywhere. Proposition 1 applies to exactly these relationships. The fast pipeline therefore loses evidence mostly at coarse levels, through second-hand summarization, and loses scope mostly at fine levels, through boundaries.

TABLE V
STRONGEST FORM IN WHICH EVIDENCE REACHES A SELECTED INPUT (%), $B = 8{,}000$; THE TIERS ARE THOSE OF SECTION III

Level	Index	In scope	Endpoint	Elsewhere	Absent
0 (root)	GraphRAG	1.9	2.4	8.1	87.6
	RGI	12.4	2.0	7.1	78.5
1	GraphRAG	7.8	30.1	23.6	38.5
	RGI	27.4	23.0	15.8	33.8
2 (default)	GraphRAG	12.4	75.8	11.4	0.4
	RGI	34.6	56.3	8.7	0.3
3	GraphRAG	12.1	84.6	3.3	0.1
	RGI	35.6	61.3	3.0	0.1

B. Selection in the standard pipeline

The fast pipeline's text-unit builder does not filter relationships, but the graph builder of the standard pipeline does. We ran GraphRAG's own `explode_communities`, `_prep_nodes`, `_prep_edges` and `build_local_context` on the census graphs. Descriptions were left as GraphRAG's own placeholder, which is safe because the selection step never reads them. As Table VI shows, only 4.6% to 16.3% of within-community relationships make it into their community's input at any level, before any budget is applied. Over all levels the builder also puts 8,702 cross-community relationships into inputs, which happens when the single record an entity contributes points to an entity outside its community.

With a mean degree of 48.1, our graphs are dense, and Proposition 3 cuts deeply. Graphs extracted by an LLM are usually much sparser, so the same code would remove fewer relationships from them, and a practitioner can estimate how many from the mean internal degree alone. The issue report [6] traces the behavior back to version 0.9.0. A fix has been proposed and linked to that issue, although it had not been merged when we wrote this paper.

TABLE VI
WITHIN-COMMUNITY RELATIONSHIPS REACHING THE STANDARD PIPELINE'S SUMMARY INPUT (GRAPHRAG 3.1.1 `BUILD_LOCAL_CONTEXT`). COMM. = COMMUNITIES; INTRA RELS = WITHIN-COMMUNITY RELATIONSHIPS; CROSS RELS IN = CROSS-COMMUNITY RELATIONSHIPS PLACED IN INPUTS

Level	Comm.	Intra rels	Reach input	Share	Cross rels in
0	123	61,415	2,841	4.6%	1,982
1	380	32,684	1,728	5.3%	3,047
2	504	14,319	897	6.3%	2,780
3	200	3,681	345	9.4%	832
4	19	370	53	14.3%	54
5	4	104	17	16.3%	7

C. Recovery at an unchanged budget

Table IV shows a higher in-scope share under RGI at every level. At the root it goes from 1.9% to 12.4%, and at level 2 from 12.4% to 34.6%; to get there at level 2, RGI writes 23,971 bridge records and 1,842 compact facts over the whole corpus. With each novel weighted equally, the level 2 gain is 24.6 points (95% CI [20.5, 28.3]). We also checked the guarantees on all 880 combinations of novel, level, budget and configuration. None of the relationships GraphRAG kept in scope was pushed out of scope by RGI (0 regressions), and Algorithm 1 never had to fall back (0 times). At $B = 8{,}000$ no input went over 8,000 tokens, and the certificate held in every run.

Absent evidence, the case covered by Proposition 1, also drops under RGI at the coarse levels; at level 0 its share falls from 87.6% to 78.5%. Level 2 had little absent evidence to begin with, so the change there is small (0.4% to 0.3%). The two-tier layout does have a cost. Of the 2,389 text units GraphRAG admits at level 2, RGI keeps 2,295 and uses the space of the others for compact facts. Some relationships were visible only incidentally, through text in the units that were dropped, and they lose that visibility. Summed over the four levels at $B = 8{,}000$ this happened 570 times, and at no level did it affect more than 0.21% of relationships. None of these relationships went out of scope.

The budget sweep in Fig. 4 shows that, at fine levels, raising the budget does less than RGI does. With only 4,000 tokens, RGI keeps 26.5% of level 2 relationships in scope, which is more than GraphRAG manages with 32,000 (14.9%). The root behaves differently because its inputs overflow, and there a larger budget does help: over the same range GraphRAG's share climbs from 0.2% to 27.7% and RGI's from 6.3% to 42.5%.

D. Do questions need the lost content?

Of the 4,371 gold triples, 99.3% could be parsed and 45.8% linked at both endpoints, while 29.8% resolved to a relationship in the graph. That puts the unlink rate at 54.2%,

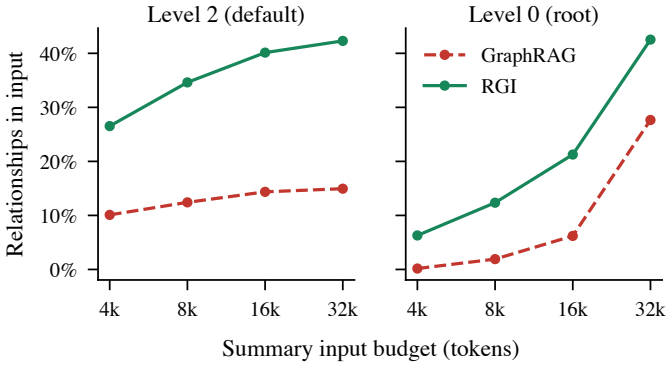


Fig. 4. In-scope share as the summary input budget grows from 4k to 32k tokens. At level 2, RGI with 4k tokens already keeps more relationships in scope than GraphRAG with 32k.

well over the 15% limit we pre-registered, and by our own rule this section therefore cannot count as gate G2. What it can do is describe the 840 questions (41.8% of the benchmark) whose gold evidence does resolve. These questions are skewed toward prominent entities, which the extractor picks up more reliably and which RGI's priority order also favors.

With that caveat, Table VII and Fig. 5 show that at level 2, 79.9% of these questions have at least one gold relationship out of scope under GraphRAG and 15.2% under RGI. The pattern holds for all four question types. Evidence tells a different story by level. At level 2, 99.8% of gold relationships have a supporting unit in some input under GraphRAG, so their loss is a loss of scope. At the root only 24.6% do, against 65.3% under RGI, and the share of these questions with at least one gold relationship whose evidence is absent everywhere falls from 79.8% to 40.7%.

TABLE VII
QUESTIONS WITH AT LEAST ONE GOLD RELATIONSHIP OUT OF SCOPE (%), AMONG THE 840 QUESTIONS WHOSE GOLD EVIDENCE RESOLVES

		$\Pr[Y(q) \cap L_\ell \neq \emptyset]$		
Slice	n	GraphRAG	RGI	Change [95% CI]
Level 0 (root)	840	95.6	48.3	−47.3 [43.9, 50.6]
Level 1	840	88.1	23.8	−64.3 [61.0, 67.5]
Level 2 (default)	840	79.9	15.2	−64.6 [61.4, 67.9]
Level 3	840	79.4	14.9	−64.5 [61.2, 67.7]
Level 2 by question type				
Complex Reasoning	317	84.5	17.0	−67.5
Contextual Summarize	183	79.8	13.7	−66.1
Creative Generation	52	98.1	15.4	−82.7
Fact Retrieval	288	71.5	14.2	−57.3

E. Sensitivity and ablations

Graph density follows from the co-occurrence window. Shrinking text units from 1,200 to 600 and 300 tokens lowers the mean degree from 48.1 to 31.0 and 19.2 (Table VIII). GraphRAG's in-scope share at level 2 rises from 12.4% to 19.9% and 27.6%, and RGI's from 34.6% to 43.6% and 57.0%, with no regressions at any size. The size of the loss depends on the graph; its existence and RGI's gain do not.

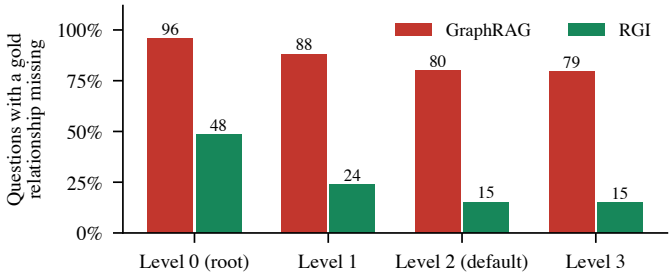


Fig. 5. Share of questions with a gold relationship out of scope, by query level, among questions whose gold evidence links to the graph.

TABLE VIII
SENSITIVITY TO TEXT-UNIT SIZE (IN-SCOPE SHARE, %, $B = 8{,}000$)

Chunk	Mean	In input, level 2		In input, level 0	
tokens	degree	GraphRAG	RGI	GraphRAG	RGI
1200 (default)	48.1	12.4	**34.6**	1.9	**12.4**
600	31.0	19.9	**43.6**	6.1	**19.8**
300	19.2	27.6	**57.0**	26.1	**41.8**

Table IX varies one parameter at a time. At level 2, where most inputs fit, bridge records carry most of the gain: without them RGI reaches only 14.7% through compact facts in the few overflowing inputs. The minimum support m matters more than the share b: moving b from 0.06 to 0.24 changes the in-scope share from 29.4% to 37.2%, while moving m from 3 to 1 changes it from 21.9% to 49.8%. At the root, bridges and compact facts compete for the same tail of the budget. Removing bridges raises the in-scope share there (13.4% against 12.4%), a lower cutoff $f = 0.6$ raises it further (21.5%) by giving compact facts more room, and without the compact tier ($f = 1.0$) RGI keeps only 2.3%. The best allocation therefore depends on the level, a point we return to in Section VII.

TABLE IX
ABLATIONS AT $B = 8{,}000$ (% OF RELATIONSHIPS). b = BRIDGE SHARE, f = FULL-ITEM CUTOFF, m = MINIMUM SUPPORT; DEFAULTS $b = 0.12$, $f = 0.8$, $m = 2$

	Level 2 (default)		Level 0 (root)	
Configuration	In input	Bridged	In input	Bridged
GraphRAG	12.4	n/a	1.9	n/a
No bridges (b=0)	14.7	0.0	13.4	0.0
b=0.06	29.4	15.0	13.2	2.6
b=0.12 (default)	34.6	20.6	12.4	4.5
b=0.24	37.2	24.5	11.1	6.4
f=0.6	35.5	20.6	21.5	4.5
f=1.0 (no compact tier)	31.2	18.8	2.3	0.4
m=1	49.8	35.8	13.0	5.6
m=3	21.9	7.7	12.2	2.8

F. From loss to answers: a controlled simulation

The census stops at the summary input. To see how loss could propagate to answers we built GraphRAG X-Ray, an interactive simulation that we also use for demonstration. It

models a synthetic index shaped like George Eliot's *Middlemarch*: 102 entities and 162 relationships taken from the novel's plot, plus a generated long tail, giving 360 entities and 905 relationships in 22 leaf communities nested into 8, 3 and 1 above them. Token costs, weights and the long tail are generated. Inputs follow GraphRAG's first-overflow rule with its priority order. Retrieval reads the top $K = 3$ summaries, ranked by entity overlap with the question, a size prior and noise shared by both methods. A stochastic judge marks an answer correct with probability 0.8, 0.45 or 0.15 when all, at least half, or less than half of the evidence was retrieved. There are 48 questions: 5 written by hand and the rest sampled as paths of two to four relationships. The simulated RGI applies the two-tier layout to every input, a simpler variant of Algorithm 1.

Table X and Fig. 6 give the outcome. RGI retrieves more evidence at every level. At the leaf level the paired difference lies between 18.0 and 35.9 points (95% bootstrap CI over questions), and at no level does any question or relationship regress (0 cases). For the original index, reading more summaries hardly helps. Moving from $K = 2$ to $K = 6$ changes leaf-level evidence retrieval only from 48.5% to 49.2%, since the lost relationships are in none of the summaries. As on the real data, the choice of which relationships to bridge matters more than the budget that bridges receive: lowering the bridge threshold from weight 5 to 3 raises retrieval from 61.4% to 89.6%, whereas raising the bridge share from 6% to 24% leaves it where it was (75.7% and 75.7%). At the root a larger budget helps both methods, and GraphRAG keeps 6.3% with 4,000 tokens and 29.9% with 16,000. That gain costs tokens on every query, while the residual store adds none. We wrote the simulator ourselves, so these figures illustrate the mechanism only under our own assumptions.

TABLE X
CONTROLLED SIMULATION (SYNTHETIC DATA, 48 QUESTIONS, %). ORIG. = ORIGINAL GRAPHRAG; NUMBERS IN PARENTHESES ARE COMMUNITIES PER LEVEL

	Retained		Evidence retrieved		Judged correct	
Level	Orig.	RGI	Orig.	RGI	Orig.	RGI
Leaf (22)	72.6	78.8	49.2	75.7	41.7	56.2
Community (8)	84.2	88.4	69.1	84.7	52.1	62.5
Higher (3)	46.3	65.2	80.2	87.0	60.4	66.7
Root (1)	14.4	23.4	54.3	71.2	43.8	50.0

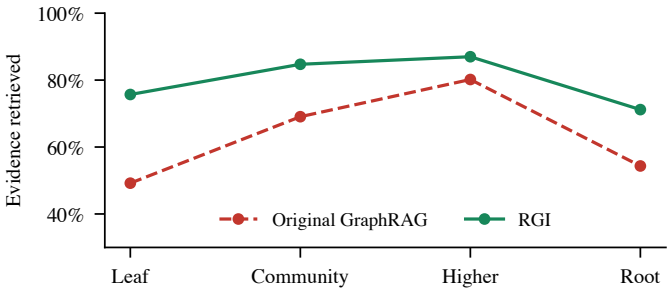


Fig. 6. Share of evidence retrieved at each level in the simulation. The data are synthetic, and the figure illustrates the mechanism only.

VII. DISCUSSION

The census separates two kinds of loss that are easy to mix up. At fine levels, boundaries take relationships out of scope, yet in the fast pipeline the text behind them usually still reaches a summarizer writing about one of the two entities, and it is then up to the model whether the report mentions the relationship. At coarse levels the problem is more serious. Summaries written from other summaries drop the evidence itself, and Proposition 1 then applies in full. A study that looks at only one of these losses will either overstate or understate the problem.

RGI works mainly as bookkeeping on top of GraphRAG. Inside the inputs where they matter it writes boundary relationships out explicitly, it brings back relationships that the budget would have dropped, and everything else goes into the residual store. Because the clustering is unchanged, multi-topic loss stays as it was. RGI also has no way of making a summarizer use the material it receives, and its compact facts hold less information than the text they replace. Where the graph has descriptions, a bridge record can include the extracted description, whereas in the fast pipeline it holds only the entity pair and its support.

How the budget should be split depends on the level. At fine levels the slack in an input costs nothing, and bridge records are the only thing RGI can add there. At the root, bridges and compact facts compete for the same space, and in our ablation compact facts gave more. An allocation that adapts to the level, or a learned salience score, would be a natural next step.

Our results also have to be squared with those of Edge et al., who found no significant difference between global search and a summarization baseline that uses no partition [3]. That baseline cannot have boundary loss, so their finding suggests that partition loss leaves aggregate answer quality on broad sensemaking questions unchanged. We make a narrower claim, measured element by element. We expect an effect on fact retrieval and on multi-hop questions whose evidence spans communities, and the protocol in Section IX tests this separately for each question type. LightRAG [13], which has no community layer at all, is competitive on GraphRAG-Bench [8]. We still think community summaries are worth keeping, because they let a reader navigate a corpus in a way that direct entity retrieval does not, and RGI is an attempt to keep them while accounting for what they cost.

VIII. THREATS TO VALIDITY

Several threats come from the way our graphs were built. GraphRAG's default extractor finds noun phrases, and we replaced it with a proper-noun extractor of our own. It produces the same output format and follows the same stop-word and token rules, but it leaves out multi-word common noun phrases. All later stages run GraphRAG's own code and the selection analysis does not depend on the extractor at all, yet the absolute shares would be different under the default one. Our graphs are also dense, with a mean degree of 48.1, because entities are linked whenever they co-occur in a 1,200-token unit, and density makes both boundary and selection

loss larger. The chunk-size sensitivity run puts a bound on this effect without eliminating it. The data are uneven as well: one novel holds 34% of all relationships, which is why every pooled figure is reported next to per-novel means with intervals.

Other threats concern what we measure. The census uses the fast pipeline, where inputs are text units, whereas in the standard pipeline they are entity and relationship descriptions written by an LLM. We can measure the selection step of the standard builder exactly, but its budget loss depends on how long those descriptions are, and only an LLM run produces them. Measuring scope also leaves two gaps. A relationship that is in an input may still be left out of the summary, and one that is out of scope may still be mentioned in it. Gate G1 measures the second gap, while the first requires the LLM run.

More than half of the gold triples do not link, so the gold analysis covers a biased subset of questions and cannot be read as a test of gate G2. The simulation is synthetic throughout, and that includes its judge. Finally, all of our results describe GraphRAG 3.1.1. If a fix for the selection behavior is merged, the findings of Section VI-B would change, although the audit and the certificate would still apply.

IX. Pre-registered Protocol for the Answer-Level Test

Everything measured so far happens at index time, and it matters in practice only if it changes the answers. We committed the thresholds in Table XI to version control on 15 August 2026 (commit `d223de8`), before any corpus was downloaded or any index built, following the logic of preregistration [32]; the linking parameters and tokenizer followed in commit `21e43d8`. Each gate is a kill criterion: if it fires, the claim is dropped and the result is reported as a null.

TABLE XI
Kill criteria committed before any data existed

Gate	Question	Stop if
G1	Do summaries already paraphrase out-of-scope relationships? Measured by natural language inference (NLI) on 300 stratified edges, once two annotators agree at Cohen's $\kappa \geq 0.7$ [33]	$> 70\%$
G2	Do questions need the lost content? $\Pr[Y(q) \cap L_\ell \neq \emptyset]$	$< 10\%$
G3	Are failures caused by the index rather than by retrieval or generation?	$< 15\%$
G4	Does restoring the lost content change answers? Oracle minus placebo accuracy, 95% CI	includes 0

The causal test (G4) compares two injections at an identical token count: the actual lost gold evidence (oracle) and randomly chosen lost relationships of the same size (placebo). Holding the count fixed separates content from length, which a before-and-after comparison cannot do. Accuracy differences are paired by question and reported with bootstrap intervals [31], McNemar's test [34] and an effect size. The recoverability check in G1 uses a cross-encoder NLI model [35] only after human agreement is established. The toolkit that runs all four gates is complete and passes 166 tests on synthetic fixtures; running it needs an LLM-built index and the two annotators.

X. Future Work

The next steps follow from the limits above: run the gates on an LLM-built index of GraphRAG-Bench Novel and replicate on its Medical corpus; compare GraphRAG, RGI, RGI with placebo residuals and LightRAG at equal tokens, by question type; implement query-time residual retrieval and measure what it adds beyond the summaries; replace the fixed bridge rule with a level-aware or learned allocation; add pointers that let multi-topic entities appear in the communities they were not filed under; and repeat the census under GraphRAG's default extractor once its data can be installed.

XI. Conclusion

Community-summary GraphRAG decides what can be known about a corpus before any question is asked. We showed where it decides: at boundaries, in the input builder, at the budget and through second-hand summaries. We measured each of these exactly with GraphRAG's own code. RGI keeps the summaries and the search as they are, spends the same budget differently, and keeps an account of every relationship. Within that budget it roughly triples the share of relationships in scope at the default level and never drops one GraphRAG kept. Whether that changes answers is the question our preregistered protocol is built to settle.

Data and Code Availability

The census, the selection analysis, the simulation and every figure and table in this paper are produced by scripts in the project repository (`paper/experiments`), from GraphRAG-Bench Novel and GraphRAG 3.1.1, without model calls. Numbers in the text are generated macros, not transcriptions.

Acknowledgment

[To be completed by the authors.]

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023.

[3] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," arXiv:2404.16130, 2024.

[4] V. A. Traag, L. Waltman, and N. J. van Eck, "From Louvain to Leiden: Guaranteeing well-connected communities," *Scientific Reports*, vol. 9, p. 5233, 2019.

[5] Microsoft, "GraphRAG: A modular graph-based retrieval-augmented generation system, version 3.1.1," https://github.com/microsoft/graphrag, 2026.

[6] E. Grenat, "[Bug]: Community report context seems to keep only one edge per node," microsoft/graphrag GitHub issue #2583, Sep. 2026.
[7] T. Wiegand, G. J. Sullivan, G. Bjøntegaard, and A. Luthra, "Overview of the H.264/AVC video coding standard," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 7, pp. 560–576, 2003.
[8] Z. Xiang *et al.*, "When to use graphs in RAG: A comprehensive analysis for graph retrieval-augmented generation," in *International Conference on Learning Representations*, 2026, arXiv:2506.05690.
[9] J. Chung, B. D. Pedigo, E. W. Bridgeford, B. K. Varjavand, H. S. Helm, and J. T. Vogelstein, "GraSPy: Graph statistics in Python," *Journal of Machine Learning Research*, vol. 20, no. 158, pp. 1–7, 2019.
[10] D. Edge, H. Trinh, and J. Larson, "LazyGraphRAG: Setting a new standard for quality and cost," Microsoft Research Blog, Nov. 2024.
[11] H. Han, L. Ma, Y. Wang, H. Shomer, Y. Lei, Z. Qi, K. Guo, Z. Hua, B. Long, H. Liu, C. C. Aggarwal, and J. Tang, "RAG vs. GraphRAG: A systematic evaluation and key insights," arXiv:2502.11371, 2025.
[12] Y. Zhou, Y. Su, Y. Sun, S. Wang, T. Wang, R. He, Y. Zhang, S. Liang, X. Liu, Y. Ma, and Y. Fang, "In-depth analysis of graph-based RAG in a unified framework," *Proceedings of the VLDB Endowment*, vol. 18, no. 13, pp. 5623–5637, 2025.
[13] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, "LightRAG: Simple and fast retrieval-augmented generation," arXiv:2410.05779, 2024.
[14] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically inspired long-term memory for large language models," in *Advances in Neural Information Processing Systems*, 2024.
[15] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR: Recursive abstractive processing for tree-organized retrieval," in *International Conference on Learning Representations*, 2024.
[16] X. He, Y. Tian, Y. Sun, N. V. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi, "G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering," in *Advances in Neural Information Processing Systems*, 2024.
[17] S. Wang, Y. Fang, Y. Zhou, X. Liu, and Y. Ma, "ArchRAG: Attributed community-based hierarchical retrieval-augmented generation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 19, 2026, pp. 15 868–15 876.
[18] B. Peng, Y. Zhu, Y. Liu, X. Bo, H. Shi, C. Hong, Y. Zhang, and S. Tang, "Graph retrieval-augmented generation: A survey," arXiv:2408.08921, 2024.
[19] H. Huang, Y. Huang, J. Yang, Z. Pan, Y. Chen, K. Ma, H. Chen, and J. Cheng, "Retrieval-augmented generation with hierarchical knowledge," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025.
[20] Y. Zhang, R. Wu *et al.*, "LeanRAG: Knowledge-graph-based generation with semantic aggregation and hierarchical retrieval," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 41, 2026, arXiv:2508.10391.
[21] Y. Li, K. Yang, T. Wang, B. Chen, B. Li, and C. Mao, "Deep GraphRAG: A balanced approach to hierarchical retrieval and adaptive integration," arXiv:2601.11144, 2026.
[22] J. Wang, H. Huang, X. Ge, J. Su, W. Liu, and S. Lian, "OMD-GraphRAG: Enhancing GraphRAG with ontology-guided extraction, multi-dimensional clustering and dual-channel fusion," arXiv:2603.25152, 2026.
[23] J. Hossain and A. E. Sarıyüce, "Core-based hierarchies for efficient GraphRAG," arXiv:2603.05207, 2026.
[24] F. Xu, W. Shi, and E. Choi, "RECOMP: Improving retrieval-augmented LMs with compression and selective augmentation," in *International Conference on Learning Representations*, 2024.
[25] T. Chen, H. Wang, S. Chen, W. Yu, K. Ma, X. Zhao, H. Zhang, and D. Yu, "Dense X retrieval: What retrieval granularity should we use?" in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 15 159–15 177.
[26] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
[27] S. Fortunato and M. Barthélemy, "Resolution limit in community detection," *Proceedings of the National Academy of Sciences*, vol. 104, no. 1, pp. 36–41, 2007.
[28] B. H. Good, Y.-A. de Montjoye, and A. Clauset, "Performance of modularity maximization in practical contexts," *Physical Review E*, vol. 81, p. 046106, 2010.
[29] S. Fortunato, "Community detection in graphs," *Physics Reports*, vol. 486, no. 3, pp. 75–174, 2010.
[30] R. Guimerà and L. A. N. Amaral, "Functional cartography of complex metabolic networks," *Nature*, vol. 433, pp. 895–900, 2005.
[31] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
[32] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proceedings of the National Academy of Sciences*, vol. 115, no. 11, pp. 2600–2606, 2018.
[33] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
[34] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
[35] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with disentangled attention," in *International Conference on Learning Representations*, 2021.



Jeyaviknan SR received the B.Tech. degree in Computer Science and Engineering from Vellore Institute of Technology, Chennai, India, in 2025. His research interests include anonymous network traffic analysis, deep learning for network security, website fingerprinting attacks and defenses, and privacy-preserving systems.



Adithya DS received the B.Tech. degree in Computer Science and Engineering from Vellore Institute of Technology, Chennai, India, in 2025. His research interests include encrypted traffic analysis, Transformer networks, sequence modeling, and network privacy and security.



Aatreye Dev received the B.Tech. degree in Computer Science and Engineering from Vellore Institute of Technology, Chennai, India, in [year]. Research interests include [research interests].



Karthika Veeramani is an Assistant Professor in the School of Computer Science and Engineering at Vellore Institute of Technology (VIT), Chennai. She received the Ph.D. and M.E. degrees in Computer Science and Engineering from Anna University, and the B.Tech. degree in Information Technology from University College of Engineering, BIT Campus, Tiruchirappalli. Before her current role, she worked for two years as a Programmer Analyst at Cognizant Technology Solutions, Chennai. She holds an Indian patent on regularized discriminant analysis and has published research papers in international journals and conferences. Her research interests include big data analytics, machine learning, and blockchain technology. She can be contacted at karthika.v@vit.ac.in.